

Run the code cell below and select all of the fastq files to upload them to the google colab directory. It may take a few minutes.

```
from google.colab import files
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving sequence_data1.fastq to sequence_data1 (?).fastq

```
!pip install biopython
from Bio import SeqIO
import matplotlib.pyplot as plt
```

Requirement already satisfied: biopython in /usr/local/lib/python3.10/dist-packages (1.83)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from biopython) (1.23.5)

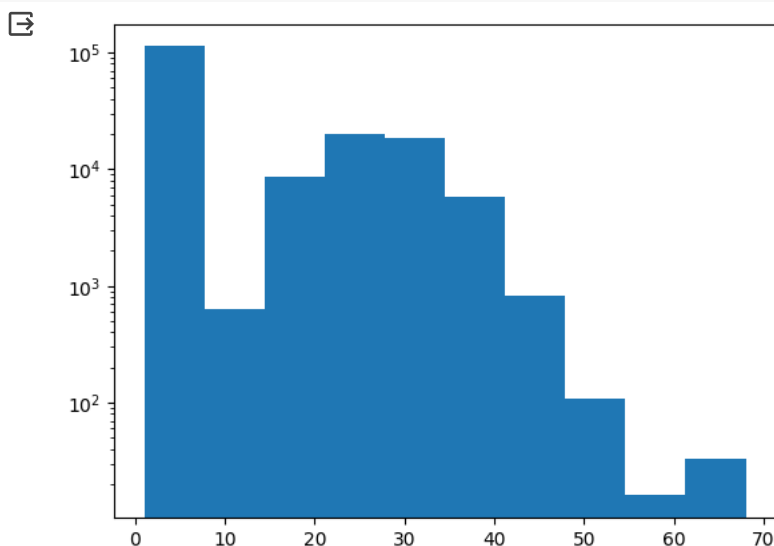
```
#write your code here for the input_fastq = "sequence_data1.fastq"
fastq_file = "sequence_data1.fastq"
output_file = "frequencies.tab"
```

```
kmer_len = 31
kmers = []
```

```
with open(fastq_file, "r") as handle:
    for record in SeqIO.parse(handle, "fastq"):
        sequence = str(record.seq)
        for i in range(len(sequence) - kmer_len + 1):
            kmer = sequence[i:i+kmer_len]
            kmers.append(kmer)
```

```
kmer_count = {}
for kmer in kmers:
    if kmer in kmer_count:
        kmer_count[kmer] += 1
    else:
        kmer_count[kmer] = 1
with open(output_file, "w") as output_handle:
    for kmer, count in kmer_count.items():
        output_handle.write(f"{kmer}\t{count}\n")
```

```
plt.hist(kmer_count.values(), bins=10)
plt.yscale("log")
```



```
#to download the files you created:
from google.colab import files
files.download('frequencies.tab')
```

2. Now run the same code with the file exercise2.fastq.gz . https://drive.google.com/file/d/1yfXxCOANzG5aHC_BHwRg-x0tzczZYed9/view?usp=drive_link

*Tip: The file is compressed in gz format (because it's a large file and we want to save space) so before open it with Biopython, open it with gzip module: `import gzip`

What do you see and how do you interpret this histogram compared to the previous one? Where are the peaks you found relative to the X axis.

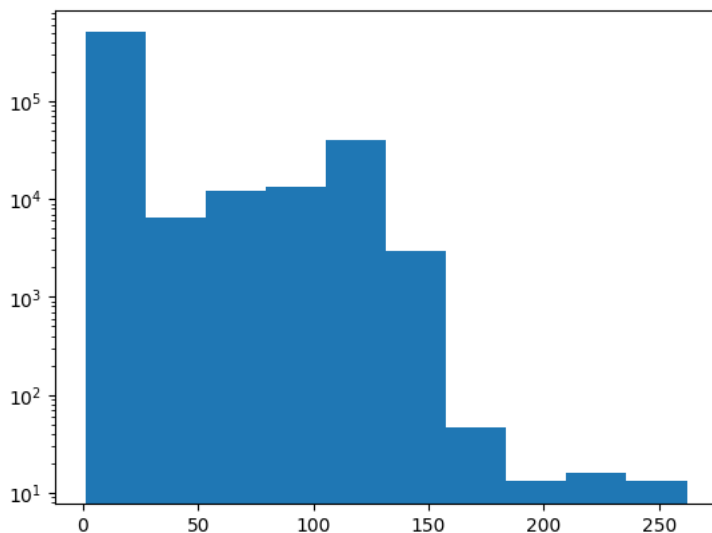
```
#write you code here for the input_fastq = "subsample.fastq.gz"
# I uploaded by unzipping the file and uploaded. fastq using same method as question 1.
fastq_file = "exercise2.fastq"
output_file = "frequencies.tab"

kmer_len = 31
kmers = []

with open(fastq_file, "r") as handle:
    for record in SeqIO.parse(handle, "fastq"):
        sequence = str(record.seq)
        for i in range(len(sequence) - kmer_len + 1):
            kmer = sequence[i:i+kmer_len]
            kmers.append(kmer)

kmer_count = {}
for kmer in kmers:
    if kmer in kmer_count:
        kmer_count[kmer] += 1
    else:
        kmer_count[kmer] = 1

plt.hist(kmer_count.values(), bins=10)
plt.yscale("log")
```



I observe two histogram each representing a set of fastq file that contain contigs of kmer values and respective counts. When setting the kmer length to 31, the histogram plots x-axis represents k-mers within the list to their respective frequencies, and y-axis represents the frequency of each kmer counts in a log scale.

Both histogram looks left-skewed and have mode frequency/peaks of kmer counts in the left-most bin (question 1: 0-7 counts(around 10 to the power of 5 frequencies) , question 2: 0-25 counts (around 10 to the power of 6 frequencies)), meaning that there are most kmers counts in less frequencies and that most kmers are unique. Moreover, looking at question 1, the x-axis of histogram holds a value of 0-70 counts with each bin representing 5 counts, the median/another peak around 28 counts, and most frequencies lies around 15-42 counts. Looking at question 2, the x-axis of histogram holds a value of 0-264 counts with each bin representing 25 counts, the median/another peak around 115 counts, and most frequencies lies around 50-150 counts. Thus, because we set the kmer length to be the same (31), we can possibly conclude that the second fastq file probably contains a longer length of genome, resulting in more counts per kmers and more frequencies per kmer counts.